Google Antigravity Customization Architecture

Enforcing Operational Rules via AGENTS.md vs. Triggering Instructions via SKILL.md Frontmatter

Executive Summary: *Antigravity separates agent customization into two complementary mechanics:* ***Operational Rules (AGENTS.md / GEMINI.md)*** *enforce persistent, ambient constraints and architectural standards hierarchically across directories, while* ***Skills (SKILL.md)*** *provide event-driven, multi-step runbooks triggered on-demand via YAML frontmatter through progressive disclosure.*

1. Rules vs. Skills: Core Architectural Comparison

Core Dimension	Operational Rules (AGENTS.md / GEMINI.md)	Triggered Skills (skills//SKILL.md)
Operational Role	**Guardrails & Invariants**: 'What the agent must / must not do'.	**Actionable Runbooks**: 'How the agent executes a specialized workflow'.
Activation Mechanism	**Ambient & Hierarchical**: Always-on within directory scope.	**On-Demand (Progressive Disclosure)**: Triggered by user prompt matching frontmatter.
Metadata Requirement	No YAML frontmatter required; pure Markdown.	**Mandatory YAML frontmatter** (name, description).
Context Cost	Injected into context when operating in that directory subtree.	Zero context overhead until description matches and skill is activated.
Directory Structure	Placed directly in repo root or subsystem subdirectories.	Structured directory: SKILL.md, scripts/, references/, examples/.

2. Deep Dive: Enforcing Operational Rules via AGENTS.md

AGENTS.md (and GEMINI.md) files establish immutable constraints for the agent. Whenever the agent reads, edits, or operates in a directory, Antigravity performs an **upward directory walk** to the repository root.

- **Hierarchical Inheritance:** A root /AGENTS.md applies globally, while /src/api/AGENTS.md adds localized API constraints.
- **Automatic Deduplication:** Even if multiple references traverse the same rules file, Antigravity resolves it once per conversation turn.
- **Ideal Declarations:** Formatting rules, forbidden APIs (e.g. 'Never use deprecated library X'), testing mandates, commit styling.

```
# Repository Operational Rules (AGENTS.md)
## Architecture & Code Standards
- Always use async/await over raw Promises in backend handlers.
- All new database queries must use the typed QueryBuilder (no raw SQL strings).
- Ensure all public functions have strict TypeScript return types and JSDoc comments.

## Safety & Security Protocols
- Never log user PII, bearer tokens, or database connection strings.
- Run `npm test` before declaring any refactoring task complete.
```

3. Deep Dive: Triggering Instructions via SKILL.md Frontmatter

Skills encapsulate complex procedural workflows. The agent uses **Progressive Disclosure**: only the YAML frontmatter is indexed initially. When the user prompt matches the description, the agent dynamically reads the body and helper resources.

- **name:** Unique lowercase identifier (e.g., database-migration, deploy-staging).
- **description (The Trigger Contract):** Critical field. Must explicitly declare *what* the skill does and *when* the agent should trigger it in third-person.
- **Modular Resources:** Relative links to scripts/, references/, and examples/ prevent context saturation.

```
---
name: deploy-kubernetes-service
description: >-
  Use this skill when the user asks to build, package, or deploy microservices to
  the Kubernetes staging or production clusters, or when debugging rollout failures.
---
# Kubernetes Service Deployment Runbook

## Deployment Workflow
1. Run pre-flight linting and container configuration check:
   [lint-k8s.sh](./scripts/lint-k8s.sh)
2. Trigger the blue-green rollout:
   `kubectl apply -k overlays/staging`
3. Verify pod health and rollout status:
   [verification-guide.md](./references/verification.md)
```

4. Architectural Synergy: Rules + Skills in Action

How They Complement Each Other:
• **AGENTS.md (The Policeman):** Enforces *'Every database schema change must include a rollback script and 0-downtime migration.'*
• **SKILL.md (The Playbook):** Triggered when the user says *'Migrate users table'* → provides the exact 4-step sequence, helper verification scripts, and failure recovery commands.

5. Best Practice Guidelines

- **Frontmatter Specificity:** Write descriptive frontmatter with explicit trigger keywords (verbs, services, tools) so the agent never misses an intended activation.
- **Lean Rules:** Keep AGENTS.md files concise. General coding wisdom already known by the LLM should be omitted to save prompt space.
- **Externalize Heavy Scripts:** Do not embed 100-line bash/python scripts in SKILL.md; place them in scripts/ and reference them via relative markdown links.